

03 · Constraints and Keys

Why this matters in interviews. "Primary key vs unique key", "What does ON DELETE CASCADE do?", "Can a foreign key be NULL?", "Can a table have two primary keys?" — constraints are asked in almost every SQL and DBMS round, and they are the reason your data stays trustworthy.

In this chapter: what a constraint is · NOT NULL · UNIQUE · PRIMARY KEY · FOREIGN KEY (and ON DELETE) · CHECK · naming constraints · managing them later · error codes · a complete worked example · the family of "keys".

3.1 What constraints are

Constraints are rules for the data in a table. They limit what can go into a column so the data stays accurate and reliable. If an action **violates** a constraint, **the action is aborted** — the statement fails and no partial change is kept.

Constraints can be given:

- **when the table is created** — CREATE TABLE , or
- **afterwards** — ALTER TABLE ... ADD CONSTRAINT .

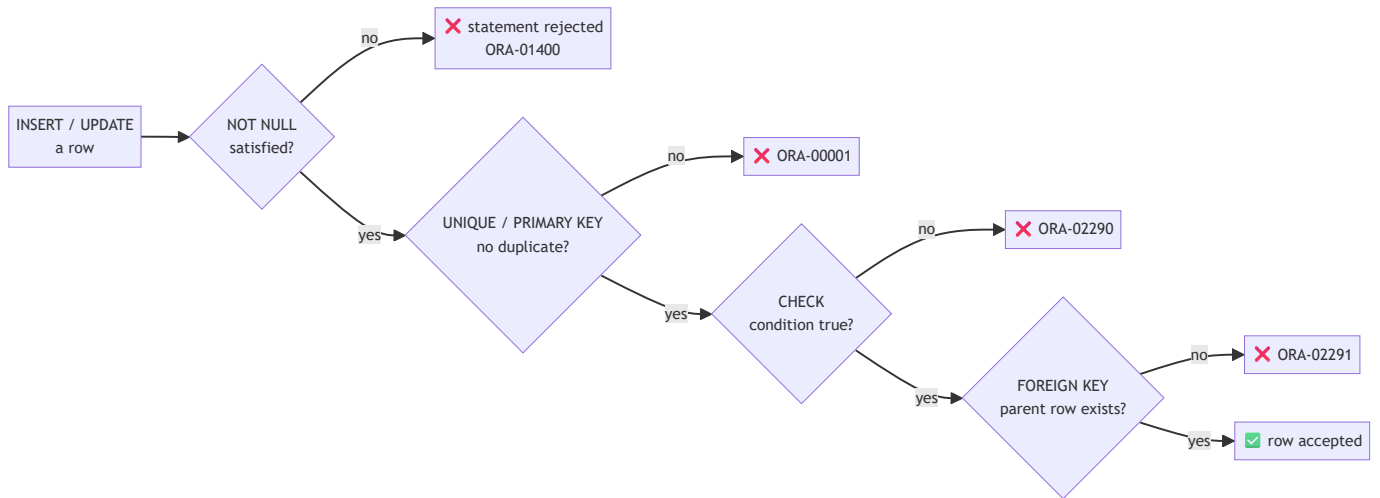
```
CREATE TABLE table_name (  
    column1 datatype constraint,  
    column2 datatype constraint,  
    column3 datatype constraint,  
    ....  
);
```

They can be **column-level** (written right after one column's data type; applies to that column) or **table-level** (written after all columns; applies to the whole table — the only way to constrain *several columns together*).

Constraint	Ensures
NOT NULL	a column cannot have a NULL value
UNIQUE	all values in a column are different
PRIMARY KEY	a combination of NOT NULL and UNIQUE — uniquely identifies each row
FOREIGN KEY	prevents actions that would destroy links between tables
CHECK	the values in a column satisfy a specific condition
DEFAULT	sets a default value when none is supplied
CREATE INDEX	used to create and retrieve data very quickly

DEFAULT and CREATE INDEX appear in the list of "commonly used" features, but in Oracle terminology **neither is a constraint** — a constraint is only NOT NULL , UNIQUE , PRIMARY KEY , FOREIGN KEY , CHECK . (This matters when you try to *name* one — see §3.8.) Indexes get Chapter 10.

Every INSERT / UPDATE passes through the checkpoint:



3.2 NOT NULL

Ensures a column **cannot have a NULL value** — use it when a column *must always have a value*.

```
CREATE TABLE Students (
  ID   INT NOT NULL,
  Name VARCHAR(50) NOT NULL
);
```

You **cannot insert a student without an ID or a Name**.

In Oracle, add it later with `MODIFY` :

```
ALTER TABLE Students MODIFY (Name NOT NULL);    -- fails with ORA-02296 if some Name is already
NULL
ALTER TABLE Students MODIFY (Name NULL);        -- takes the rule off again
```

3.3 UNIQUE

Ensures **all values in a column are different**. A table can have **many** `UNIQUE` constraints.

```
CREATE TABLE Employees (
  EmpID INT,
  Email  VARCHAR(100) UNIQUE
);
```

You **cannot insert two employees with the same email**.

- `UNIQUE` **allows NULL** (unlike a primary key). In Oracle you can store **several NULLs** in a single-column unique constraint, because NULL is never *equal* to NULL.
- Oracle enforces `UNIQUE` (and `PRIMARY KEY`) by automatically creating a **unique index** — that is why they are also fast to search.
- A **composite** unique constraint makes the *combination* unique: `UNIQUE (first_name, last_name, dob)`.

3.4 PRIMARY KEY

A combination of **NOT NULL** + **UNIQUE**. Each table can have **only one primary key**, used to **uniquely identify rows**.

```
CREATE TABLE Courses (  
  CourseID INT PRIMARY KEY,  
  CourseName VARCHAR(100)  
);
```

Every `CourseID` must be **unique and not null**.

Composite primary key

When one column is not enough, the key is made from **several columns together** — table-level syntax is required:

```
CREATE TABLE Enrollment (  
  StudentID INT,  
  CourseID INT,  
  PRIMARY KEY (StudentID, CourseID)      -- the PAIR must be unique  
);
```

A student may appear many times and a course may appear many times, but the same (*student, course*) pair only once.

Primary key vs unique key

	Primary key	Unique key
How many per table	one	many
NULL allowed	✗ never	✓ yes (Oracle: several)
Purpose	the row's identity	any other alternative identifier (email, PAN, phone)
Index created	unique index	unique index
Can be referenced by a foreign key	✓	✓

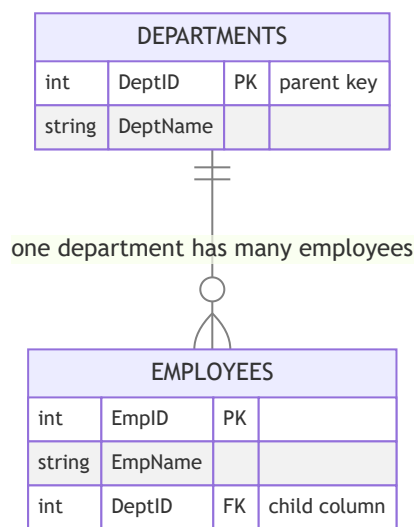
3.5 FOREIGN KEY

A foreign key **links two tables**. A column (or columns) in the **child** table refers to the primary/unique key of a **parent** table, and the database **prevents invalid data** from entering the child.

```
CREATE TABLE Departments (
  DeptID INT PRIMARY KEY,
  DeptName VARCHAR(50)
);

CREATE TABLE Employees (
  EmpID INT PRIMARY KEY,
  EmpName VARCHAR(50),
  DeptID INT,
  FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)
);
```

An employee can only be assigned to a department that exists.



Parent / child vocabulary. The table holding the **referenced** key is the *parent* (`Departments`); the table holding the **foreign key** is the *child* (`Employees`). You insert into the parent first; you delete from the child first.

Three ways to declare a foreign key

1 · In CREATE TABLE, table-level (works everywhere):

```
CREATE TABLE Orders (
  OrderID INT PRIMARY KEY,
  CustomerID INT,
  FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);
```

`CustomerID` in `Orders` must match an existing `CustomerID` in `Customers` .

2 · Inside the column definition (column-level shorthand):

```
CREATE TABLE Orders (
  OrderID INT PRIMARY KEY,
  CustomerID INT REFERENCES Customers(CustomerID)
);
```

Shorter — and it can carry a name in Oracle too: `CustomerID INT CONSTRAINT fk_ord_cust REFERENCES Customers(CustomerID)`.

3 · After the table exists, with `ALTER TABLE` — useful when tables are created separately or you want to add the key later:

```
ALTER TABLE Orders
ADD CONSTRAINT fk_customer
FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID);
```

`fk_customer` is the **name of the foreign-key constraint**.

ON DELETE / ON UPDATE — what happens to the children?

These clauses define **what happens in the child table when a referenced parent record is deleted or updated**.

Action	Description
CASCADE	automatically deletes / updates the child rows when the parent row is deleted / updated
SET NULL	sets the foreign key to NULL if the parent row is deleted / updated
SET DEFAULT	sets the foreign key to its default value
NO ACTION	prevents the deletion / update — the default behaviour
RESTRICT	same as <code>NO ACTION</code> , but the check is done immediately

```
CREATE TABLE Orders (
  OrderID INT PRIMARY KEY,
  CustomerID INT,
  FOREIGN KEY (CustomerID)
    REFERENCES Customers(CustomerID)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);
```

If a customer is deleted or updated in `Customers`, the corresponding rows in `Orders` are also deleted or updated automatically.

Oracle vs others. Oracle supports **only** `ON DELETE CASCADE` and `ON DELETE SET NULL` (or nothing, which behaves like `NO ACTION`). It has **no** `ON UPDATE clause`, **no** `SET DEFAULT` and **no** `RESTRICT keyword`. (Also MySQL/InnoDB accepts `SET DEFAULT` in the grammar but rejects it.) Since primary keys should never change, "ON UPDATE CASCADE" is rarely needed.

See each behaviour with real data. Parent `customers` = Alice (1), Bob (2). Child `orders`: order 1 → Alice, order 2 → Alice, order 3 → Bob. Now **delete Alice**.

Default (NO ACTION) — the delete is blocked:

```
DELETE FROM customers WHERE customer_id = 1;
```

Result — the statement is rejected

```
ORA-02292: integrity constraint (SCOTT.FK_ORD_CUST) violated - child record found
```

Table after

order_id	customer_id
1	1
2	1
3	2

ON DELETE CASCADE — Alice's orders vanish with her:

```
DELETE FROM customers WHERE customer_id = 1;
```

Table after

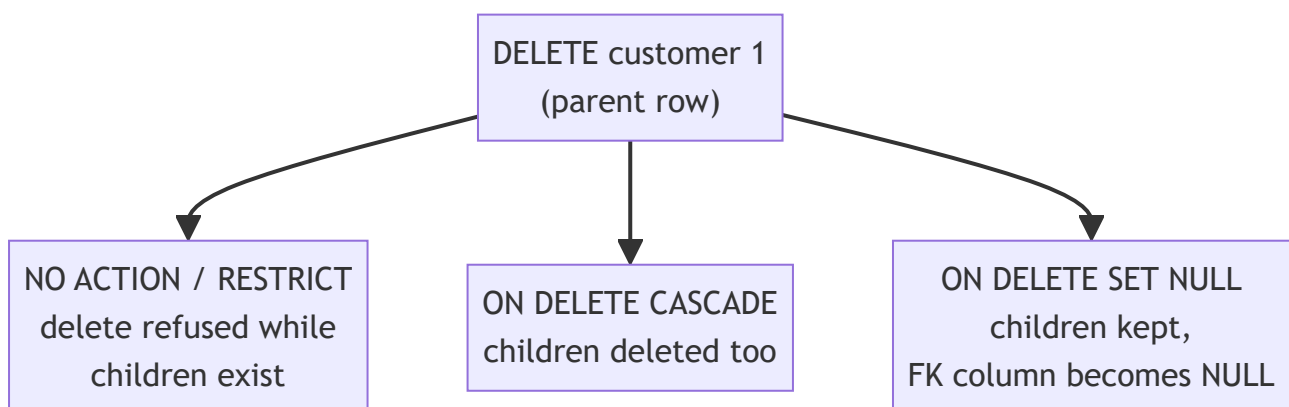
order_id	customer_id
3	2

ON DELETE SET NULL — the orders survive but lose their owner:

```
DELETE FROM customers WHERE customer_id = 1;
```

Table after

order_id	customer_id
1	NULL
2	NULL
3	2



Composite foreign keys

Used when the referenced table has a **composite** primary key. The child must supply *all* the columns:

```
CREATE TABLE Enrollment (  
    StudentID INT,  
    CourseID INT,  
    PRIMARY KEY (StudentID, CourseID),  
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),  
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)  
);  
  
CREATE TABLE Grades (  
    StudentID INT,  
    CourseID INT,  
    Grade CHAR(1),  
    FOREIGN KEY (StudentID, CourseID)  
        REFERENCES Enrollment(StudentID, CourseID) -- a two-column foreign key  
);
```

`Enrollment` is the child of `Students` and `Courses`, **and** the parent of `Grades`.

Rules and notes

- The **referenced column(s) must be a PRIMARY KEY or UNIQUE** in the parent table.
- Foreign keys **can accept NULL**, unless declared `NOT NULL`. A NULL foreign key is simply "not linked to anything" and is not checked. (For a *composite* foreign key, Oracle skips the check if **any** of its columns is NULL.)
- You **cannot delete a referenced parent row** while child rows exist — unless you defined `CASCADE` or `SET NULL`.
- **Oracle does not index foreign-key columns for you.** Add an index on the child's FK column, or joins and parent deletes will scan the whole child table (and can lock it).
- A table may reference **itself** (`manager_id` → `emp_id`) — the "self-referencing" foreign key used for hierarchies (Chapter 07).

3.6 CHECK

Ensures the values in a column **satisfy a condition**.

```
CREATE TABLE Products (  
    ProductID INT PRIMARY KEY,  
    Price DECIMAL(10, 2),  
    CHECK (Price > 0)  
);
```

You **can't insert a product with a price of 0 or negative**.

More forms of `CHECK` :

```

salary      NUMBER CHECK (salary BETWEEN 20000 AND 200000)
job_title    VARCHAR2(100) CHECK (job_title IN ('Manager','Analyst','Clerk','HR','Developer'))
phone_number VARCHAR2(100) CHECK (phone_number LIKE '____-____-____')    -- each _ is exactly one
character
commission   NUMBER CHECK (commission BETWEEN 1 AND 50)
gender       CHAR(1) CHECK (gender IN ('F','M'))
dept_name    VARCHAR2(50) CHECK (REGEXP_LIKE(dept_name, '^[A-Z][a-z]+$'))    -- one capitalised word

```

Trap — CHECK lets NULL through. A `CHECK` fails only when its condition is **FALSE**. If the column is `NULL` the condition is `UNKNOWN` and the row is **accepted**. So `CHECK (Price > 0)` still allows a `NULL` price — combine it with `NOT NULL` if a value is mandatory.

Oracle restrictions on `CHECK` : it may refer only to **columns of the same row** — no subqueries, no other tables, and no non-deterministic functions such as `SYSDATE` , `SYSTIMESTAMP` , `USER` or `ROWNUM` . (Use a trigger for rules like "hire date cannot be in the future".)

3.7 DEFAULT

Sets a default value **if no value is specified** on insert.

```

CREATE TABLE Users (
    UserID INT PRIMARY KEY,
    Country VARCHAR(50) DEFAULT 'India'
);

```

If `Country` is not given, `'India'` is inserted automatically. (More on defaults in Chapter 02.)

3.8 Naming constraints

If you don't name a constraint Oracle invents one (`SYS_C0012345`) — and that is the name you will see in an error message at 2 a.m. **Name them** so errors and `ALTER TABLE ... DROP CONSTRAINT` are readable. A common convention: `pk_` , `fk_` , `uq_` , `ck_` , `nn_` + table/column.

Kind	Inline (column-level) named?	Table-level named?
PRIMARY KEY	✓ <code>id NUMBER CONSTRAINT pk_emp PRIMARY KEY</code>	✓
UNIQUE	✓ <code>email VARCHAR2(100) CONSTRAINT uq_email UNIQUE</code>	✓
NOT NULL	✓ <code>name VARCHAR2(50) CONSTRAINT nn_name NOT NULL</code>	✗ (only inline / <code>MODIFY</code>)
CHECK	✓ <code>salary NUMBER CONSTRAINT ck_sal CHECK (salary > 0)</code>	✓

Kind	Inline (column-level) named?	Table-level named?
FOREIGN KEY	✅ <code>dept_id NUMBER CONSTRAINT fk_dept REFERENCES department(dept_id)</code>	✅ <code>CONSTRAINT fk_dept FOREIGN KEY (dept_id) REFERENCES department(dept_id)</code>
DEFAULT	❌ cannot be named — it is not a constraint	❌

Rule of thumb. The `CONSTRAINT` name keyword is used **only** for `PRIMARY KEY`, `UNIQUE`, `NOT NULL`, `CHECK` and `FOREIGN KEY`. A constraint that involves **more than one column** (composite key, or a check across two columns) **must be written at table level**. Constraint names must be unique within a schema.

⚠️ A widely-copied snippet shows `location VARCHAR2(50) CONSTRAINT df_loc DEFAULT 'Unknown' NOT NULL`. That is a **syntax error in Oracle**. The correct form is `location VARCHAR2(50) DEFAULT 'Unknown' CONSTRAINT nn_loc NOT NULL` — default first, then the (named) constraint.

3.9 A complete worked example (Oracle)

A `department` parent and an `employee` child using **every** kind of constraint. Drop order matters — child first:

```

DROP TABLE employee;
DROP TABLE department;

CREATE TABLE department (
    dept_id    NUMBER
        CONSTRAINT pk_name_dept PRIMARY KEY,
    dept_name  VARCHAR2(100)
        CONSTRAINT uq_name_dept UNIQUE
        CONSTRAINT nn_name_dept NOT NULL,
    location   VARCHAR2(100) DEFAULT 'Unknown',
    budget     NUMBER
        CONSTRAINT chk_budget_dept CHECK (budget BETWEEN 1000 AND 1000000)
);

CREATE TABLE employee (
    emp_id     NUMBER
        CONSTRAINT pk_id_emp PRIMARY KEY,
    emp_name   VARCHAR2(100)
        CONSTRAINT nn_name_emp NOT NULL,
    salary     NUMBER DEFAULT 30000
        CONSTRAINT chk_salary_emp CHECK (salary BETWEEN 20000 AND 200000),
    hire_date  DATE DEFAULT SYSDATE,
    email      VARCHAR2(100)
        CONSTRAINT uq_email_emp UNIQUE,
    dept_id    NUMBER,
    CONSTRAINT fk_id_emp FOREIGN KEY (dept_id) REFERENCES department (dept_id),
    job_title  VARCHAR2(100)
        CONSTRAINT chk_title_emp CHECK (job_title IN
('Manager', 'Analyst', 'Clerk', 'HR', 'Developer')),
    phone_number VARCHAR2(100)
        CONSTRAINT chk_number_emp CHECK (phone_number LIKE '____-____-____'),
    commission NUMBER
        CONSTRAINT chk_commission_emp CHECK (commission BETWEEN 1 AND 50)
);

```

Reading it: one column can carry **several** constraints (dept_name is both UNIQUE and NOT NULL); DEFAULT sits before the constraint; the foreign key is declared table-level in the middle of the column list (Oracle accepts that ordering); every constraint has a name you can find in USER_CONSTRAINTS .

Then the queries the constraints protect:

```

INSERT INTO employee (emp_id, emp_name, salary) VALUES (1, 'Kavi', 15000);
-- ORA-02290: check constraint (SCOTT.CHK_SALARY_EMP) violated      (salary below 20000)

INSERT INTO employee (emp_id, emp_name, dept_id) VALUES (2, 'Mala', 99);
-- ORA-02291: integrity constraint (SCOTT.FK_ID_EMP) violated - parent key not found

INSERT INTO employee (emp_id, emp_name, email) VALUES (3, 'Nila', 'n@x.com'); -- succeeds
INSERT INTO employee (emp_id, emp_name, email) VALUES (4, 'Omar', 'n@x.com'); -- same email
-- ORA-00001: unique constraint (SCOTT.UQ_EMAIL_EMP) violated

INSERT INTO employee (emp_id, salary) VALUES (5, 40000);
-- ORA-01400: cannot insert NULL into ("SCOTT"."EMPLOYEE"."EMP_NAME")

```

3.10 Adding, dropping and switching constraints

```

ALTER TABLE employee ADD CONSTRAINT ck_sal_pos CHECK (salary > 0);
ALTER TABLE employee ADD CONSTRAINT uq_phone UNIQUE (phone_number);
ALTER TABLE employee ADD (manager_id NUMBER); -- a self-referencing key needs the column
first
ALTER TABLE employee ADD CONSTRAINT fk_mgr FOREIGN KEY (manager_id) REFERENCES employee (emp_id);

ALTER TABLE employee DROP CONSTRAINT ck_sal_pos;
ALTER TABLE employee DROP PRIMARY KEY; -- add CASCADE to also drop foreign keys
pointing at it

ALTER TABLE employee DISABLE CONSTRAINT fk_id_emp; -- keep the rule, stop enforcing (bulk
loads)
ALTER TABLE employee ENABLE CONSTRAINT fk_id_emp;
ALTER TABLE employee RENAME CONSTRAINT fk_id_emp TO fk_emp_dept;

```

Deferrable constraints (Oracle): ... DEFERRABLE INITIALLY DEFERRED postpones the check to COMMIT time, so you can insert child rows before the parent within one transaction.

Find them:

```

SELECT constraint_name, constraint_type, search_condition, status
FROM   user_constraints
WHERE  table_name = 'EMPLOYEE';
-- constraint_type:  P = primary key · U = unique · R = referential (foreign key) · C = check or NOT
NULL

```

Error codes worth memorising

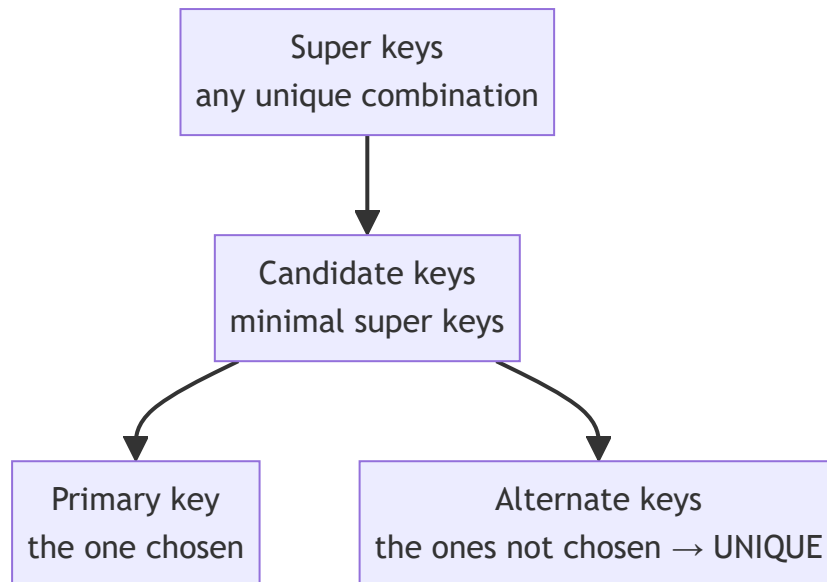
Error	Meaning	Caused by
ORA-01400	cannot insert NULL into (...)	NOT NULL

Error	Meaning	Caused by
ORA-00001	unique constraint (...) violated	duplicate PRIMARY KEY / UNIQUE
ORA-02290	check constraint (...) violated	CHECK
ORA-02291	integrity constraint violated – parent key not found	inserting a child with no parent
ORA-02292	integrity constraint violated – child record found	deleting a parent that has children
ORA-02296	cannot enable (...) – null values found	adding NOT NULL to a column holding NULLs
ORA-02266	unique/primary keys in table referenced by enabled foreign keys	TRUNCATE of a parent table

Remember the pair: **02291 = no parent (going up)**, **02292 = has children (going down)**.

3.11 The family of "keys"

Key	Meaning	Example in <code>student(id, email, phone, name, dept_id)</code>
Super key	any set of columns that identifies a row uniquely (may contain extra columns)	{id}, {id, name}, {email, phone} ...
Candidate key	a <i>minimal</i> super key — no column can be removed	{id}, {email}
Primary key	the candidate key chosen as the main identifier (NOT NULL , one per table)	id
Alternate key	candidate keys not chosen as primary	email (enforced with UNIQUE)
Composite key	a key made of two or more columns	(StudentID, CourseID) in Enrollment
Foreign key	column(s) referencing another table's primary/unique key	dept_id → department.dept_id
Surrogate key	a meaningless generated id (identity/sequence)	id NUMBER GENERATED ALWAYS AS IDENTITY
Natural key	a real-world value that is already unique	email , passport no.



Interview corner

Q. Primary key vs unique key? Both enforce uniqueness and create an index. A table has **one** primary key (never NULL) but **many** unique keys (NULL allowed). PK = the row's identity.

Q. Can a table have more than one primary key? No — but the single primary key can be **composite** (several columns).

Q. Can a foreign key be NULL? Can it be duplicated? Yes to both. NULL means "no parent"; duplicates are normal (many children per parent). What it cannot be is a non-NULL value that has no matching parent.

Q. What must a foreign key reference? A `PRIMARY KEY` or `UNIQUE` column(s) of the parent table.

Q. What does `ON DELETE CASCADE` do? And `SET NULL`? `CASCADE` deletes the child rows with the parent; `SET NULL` keeps them and nulls the foreign key. Without either, the delete is refused (`ORA-02292`).

Q. Which order do you create / drop parent and child tables in? Create the parent first; drop the child first (or `DROP ... CASCADE CONSTRAINTS`).

Q. Does `CHECK (salary > 0)` reject a NULL salary? No — the condition is UNKNOWN for NULL, which counts as *not violated*. Add `NOT NULL` .

Q. Can you name a `DEFAULT`? No, `DEFAULT` is not a constraint.

Q. Foreign key vs join? A foreign key is a **rule** stored in the schema that keeps two tables consistent; a join is a **query** that combines their rows. You can join without a foreign key.

Quick recap

- Constraints = rules; a violating statement is **aborted**. Column-level or table-level; at `CREATE` or via `ALTER` .
- `NOT NULL` , `UNIQUE` (many, NULL ok), `PRIMARY KEY` (= `NOT NULL` + `UNIQUE` , **one**, may be composite), `FOREIGN KEY` (→ PK/UNIQUE of the parent), `CHECK` (NULL passes!), `DEFAULT` (not a constraint).
- FK actions: `NO ACTION` (default) · `CASCADE` · `SET NULL` — Oracle has **no `ON UPDATE`** and no `SET DEFAULT` .
- Name your constraints; a multi-column constraint must be table-level; inline FK **is** allowed and can be named.

- Oracle does **not** auto-index foreign keys.
- Errors: 00001 dup · 01400 NULL · 02290 check · 02291 no parent · 02292 has children.